The ai-construct Computer

> For developers integrating ai-construct, and for the AI running on it.

What is this?

An ai-construct is an AI persona with its own computer. The computer has a persistent filesystem, a package manager, a scheduler, and access to your product through typed APIs. The AI writes TypeScript to do everything: read files, call product APIs, build tools for itself, schedule work, and grow its capabilities over time.

The AI has **one tool:** `preview_exec`. It writes TypeScript. The code runs in a preview world first. Filesystem changes go through a forked VFS. DB-backed product API calls run inside a real transaction cycle that rolls back at the end of preview.

The core idea

There are three layers:

1. **Commands** for computer operations like reading files, listing directories, opening context, and package management.
2. **Typed product bindings** for your application's APIs.
3. **One top-level transaction cycle** for request/invocation-scoped DB access and staged effects.

The AI writes code against commands and bindings. It does not see your raw DB driver, secrets, or internal services.

For the developer: what your AI can do

Your AI reads and writes files

The AI has a persistent Linux-like filesystem. It remembers things by writing them down.

```
const notes = await command('read', '/home/user/context/goals.md');
await command(
  'write',
```

```
  '/home/user/context/state/progress.json',
  JSON.stringify({ week: 12, streak: 5 }),
);
```

Your AI calls your product APIs

You expose product capabilities as typed bindings, but the real shared application logic lives in router-style handlers/providers.

```
export const searchExercisesHandlerProvider = createApiRouteHandlerProvider(
  (deps: { exerciseDal: ExerciseDal; effects: EffectContext; logger: Logger }) =>
    createApiRouteHandler({
      route: searchExercisesRoute,
      handler: async (args, ctx) => {
        const auth = requireTrainerAuth(ctx);
        const result = await deps.exerciseDal.find(deps.logger, {
          filter: { workspaceId: auth.workspaceId, query: args.body.query },
          limit: 20,
        });
        if (result.isErr()) throw result.error;
        return SerializableResult.toOk(result.value, StatusCode.OK);
      },
    }),
);

export const searchExercises = defineCodeBinding(
  async function searchExercises(input: SearchExercisesInput, { deps }) {
    return deps.handlers.searchExercises.execute(
      { body: input, pathParams: {}, pathQuery: {} },
      deps.executeContext,
    );
  },
);
```

The AI uses it like a normal library:

```
import { searchExercises } from 'exercise-api';

const results = await searchExercises({
  query: 'lower back',
  level: 'beginner',
```

```
});

show(results);
```

Your AI builds its own tools

The AI can write, publish, and install command packages. Over time, it builds a toolbox of reusable operations tailored to the user.

```
await command('pm', 'init', '@commands/weekly-report');
await command('write', '/home/user/scripts/@commands/weekly-report/index.ts', `
import { defineCommand } from 'sys';
import { getWorkouts } from 'workout-api';

export default defineCommand({
  name: 'weekly-report',
  description: 'Generate a weekly workout summary',
  args: {
    weeks: { type: 'number', flag: '--weeks', default: 1 },
  },
  fn: async (parsed) => {
    const workouts = await getWorkouts({ since: parsed.weeks + 'w' });
    return { total: workouts.length };
  },
  render: (result) => `${result.total} workouts`,
});
`);
await command('pm', 'publish', '@commands/weekly-report');
await command('pm', 'add', '@commands/weekly-report');
```

Your AI can call LLMs inside its code

A lightweight AI SDK is available for structured extraction, web search, and text generation — with the budget and models you configure.

```
import { ai } from 'sys/ai';
import { z } from 'zod';

const parsed = await ai.extract(userMessage, z.object({
  exercise: z.string(),
  sets: z.number(),
```

```
  reps: z.number(),
}), { model: 'fast' });
```

Your AI manages its own context window

The AI decides what to keep in attention by opening and pinning files.

```
await command('open', '/home/user/context/state/streak.json');
await command(
  'pin',
  '/home/user/context/goals.md',
  '--phase',
  'awake',
  '--reason',
  'daily reference',
);
```

For the developer: how to integrate

1. Write shared handlers/providers

Use the new `@inkibra/router` style. Provider `deps` receive invocation-scoped DALs, effects, and logger. Handler `ctx` stays auth/request context.

```
export const logWorkoutHandlerProvider = createApiRouteHandlerProvider(
  (deps: { workoutDal: WorkoutDal; effects: EffectContext; logger: Logger }) =>
    createApiRouteHandler({
      route: logWorkoutRoute,
      handler: async (args, ctx) => {
        const auth = requireTrainerAuth(ctx);
        const workout = {
          id: crypto.randomUUID(),
          trainerId: auth.trainerId,
          ...args.body,
        };

        await deps.workoutDal.insert(deps.logger, workout);

        await deps.effects.call({
```

```
          kind: 'notify-workout-logged',
          payload: { workoutId: workout.id },
          preview: 'Notify the user that their workout was logged.',
        });

        return SerializableResult.toOk({ id: workout.id, logged: true },
StatusCode.OK);
      },
    }),
);
```

2. Wrap handlers as AI bindings

Use `defineCodeBinding` in `.tool.ts` files. Build-pack generates validators and declarations automatically.

```
export const logWorkout = defineCodeBinding(
  async function logWorkout(input: LogWorkoutInput, { deps }) {
    return deps.handlers.logWorkout.execute(
      { body: input, pathParams: {}, pathQuery: {} },
      deps.executeContext,
    );
  },
);
```

3. Group bindings into an AI computer module

```
export const trainerModule = defineAiComputerModule({
  name: 'trainer',
  bindings: {
    logWorkout,
    searchExercises,
  },
  createHandlers: ({ driver, effects, logger }) => {
    const deps = createTrainerDeps({ driver, effects, logger });
    return {
      logWorkout: logWorkoutHandlerProvider(deps),
      searchExercises: searchExercisesHandlerProvider(deps),
    };
```

```
  },
});
```

4. Configure the construct

```
const construct = createConstruct({
  computer: {
    modules: [trainerModule],
    commands: [deployCommand],
    preinstalled: ['zod', 'date-fns'],
    createExecuteContext: async ({ construct, impulse }) => ({
      auth: {
        trainerId: construct.ownerId,
        workspaceId: construct.workspaceId,
      },
      requestMeta: {
        constructId: construct.id,
        impulseId: impulse.id,
      },
    }),
  },
});
```

For the AI: how to use your computer

You have one tool: `preview_exec`

Write TypeScript. Use `command()` for file and system operations. Use imports for product APIs and libraries. Use `show()` to inspect values while continuing to work with them.

Quick reference

```
const content = await command('read', '/home/user/notes.md');
await command('write', '/home/user/output.md', reportText);
show(await command('list', '/home/user/context/'));

import { getWorkouts, logWorkout } from 'workout-api';
const workouts = await getWorkouts({ since: '7d' });
```

```
await logWorkout({ exercise: 'squat', sets: 3, reps: 10, weight: 135 });

import { ai } from 'sys/ai';
import { z } from 'zod';
const parsed = await ai.extract(text, z.object({ value: z.string() }), {
  model: 'fast',
});
```

`show()`

`show()` pretty-prints a value for the model to read and returns the raw value to your code.

```
const entries = show(await command('list', '/home/user/context/'));
const recent = entries.filter((e) => e.modified > lastWeek);
```

Transaction and effect model

DB-backed product APIs run inside one top-level transaction cycle per request or AI invocation.

- **Preview:** run real code, then roll back
- **Commit / HTTP:** run real code, commit, then flush effects

Effects are fire-and-forget. They can return preview text during execution, but they do not return business data to handlers. Real delivery happens only after a successful commit. Starting a workflow is one kind of effect.

This gives the AI a real preview of database behavior without committing state.

Why this architecture

One tool, not many. The AI never wastes tokens choosing between tools.

Commands, not a shell. `command('read', path)` is precise and avoids shell syntax confusion.

`show()` over `console.log`. The AI needs to inspect data and keep using it.

Shared handlers across HTTP and AI. The same router handler/provider can back your web APIs and your AI computer.

Preview is real execution with rollback. DB-backed previews use a real transaction cycle; filesystem previews use `OverlayFs.fork()`.

Effects are staged. Side effects are explicit, fire-and-forget, and only flush after commit.

The AI builds its own tools. Commands plus `pm` let capability accumulate as code instead of staying trapped in the prompt window.